

DCE — the working manual

August 17, 2026

CONTENTS

DCE — the working manual	
Part 0 — Orientation	
Where everything lives	
The two-sentence mental model	
The rule that governs every change	
Part 1 — You have a pile of documents. Start here.	
1.1 Bulk triage	
1.2 The five piles, and what each one means	
1.3 A document that classified correctly is not “done”	
Part 2 — Adding a doctype the registry does not have	
2.1 First: is it really new?	
2.2 The anatomy of a doctype	
2.3 Choosing anchors — the part that decides everything	
Non-decisive anchors — most of them	
Decisive anchors — near-proof, and heavily policed	
Zones — read this before using one	
Negative anchors and confusables	
2.4 The checklist	
Part 3 — A document abstained and should not have	
3.1 Are zones the problem?	
3.2 Read which gate failed	

3.3 The fix is almost always an anchor

3.4 What not to do

Part 4 — A document was classified WRONG

4.1 Find the anchor that did it

4.2 If a DECISIVE anchor fired wrongly, that anchor is the bug

4.3 Then re-run everything

Part 5 — Extraction

5.1 The anatomy of a field

5.2 The six locators

5.3 The fourteen validators

5.4 Recipes

A field comes back empty

A field binds the WRONG value

Adding a field to an existing doctype

Testing extraction

Part 6 — Reading problems

The document came back needs_ocr

The document came back truncated

Policy

Part 7 — The measurement loop, which is not optional

Part 8 — Reference

Categories

Field types

Zones

Traps

Part 9 — Order of work, for a large pile

Two things worth knowing before you start

DCE — the working manual

Extending coverage, raising precision, and fixing extraction, against real documents

For: Nilendu · 17 Aug 2026

You have a pile of real documents inside the company network. Some are types the registry knows; some are not. This is the manual for turning that pile into coverage and precision, one document at a time, without breaking what already works.

Everything here is checkable against the code. File paths and commands are real.

Part 0 — Orientation

Where everything lives

What	Where
Doctype definitions	<code>dce/registry/{usa,canada,mexico,crosscountry}.py</code>
Loader and its rules	<code>dce/registry/loader.py</code>
Classification cascade	<code>dce/classify/cascade.py</code>
Extraction resolver	<code>dce/extract/resolve.py</code>
Field locators	<code>dce/extract/locators/</code>
Validators	<code>dce/extract/validate.py</code>
Corpus	<code>corpus/<cc>/</code> + <code>manifest.jsonl</code>
Bundles	<code>corpus/bundles/</code> + <code>bundles.jsonl</code>
Measurement harness	<code>tools/corpus_test.py</code>

The two-sentence mental model

Classification asks *what is this document?* It reads printed issuer text — anchors and vocabulary — and abstains unless four gates hold.

Extraction asks *what values does it contain?* It runs only after classification succeeded, against the field list on that one doctype.

They fail differently and are fixed differently. Never debug them together.

The rule that governs every change

A wrong answer is a compliance incident. An abstention is a human's afternoon.

Every threshold, every gate and every default in this system follows from that. When you are choosing between “it might catch more” and “it might be wrong more”, choose neither until you have measured, then choose precision.

Part 1 — You have a pile of documents. Start here.

1.1 Bulk triage

Do not open documents one at a time. Run the pile and let it sort itself.

Save this as `tools/triage.py`:

```
#!/usr/bin/env python3
"""Sort a directory of real documents into work piles."""
import sys, json
from pathlib import Path
from collections import Counter

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from dce.ingest.pipeline import ingest
from dce.classify.cascade import classify

root = Path(sys.argv[1])
rows, tally = [], Counter()

for f in sorted(root.rglob("*")):
    if f.is_dir() or f.name.startswith("."):
        continue
    try:
        r = ingest(f.read_bytes(), doc_id=f.name)
    except Exception as exc:
        rows.append((f.name, "ERROR", type(exc).__name__, "", ""));
    tally["ERROR"] += 1
    continue
    if r.view is None:
        rows.append((f.name, "NEEDS_OCR", r.reason[:60], "", ""));
    tally["NEEDS_OCR"] += 1
    continue
    c = classify(r.view)
    pile = "ABSTAINED" if c.abstained else "ANSWERED"
    tally[pile] += 1
    if r.truncated:
        tally["TRUNCATED"] += 1
    rows.append((f.name, pile, c.doctype_id, f"{c.confidence:.3f}",
                f"{r.text_source} pages={r.page_count} trunc=
                {r.truncated}"))

print(f"{'file':44s} {'pile':10s} {'doctype':26s} {'conf':>6s} detail")
for r in rows:
    print(f"{r[0][:44]:44s} {r[1]:10s} {r[2][:26]:26s} {r[3]:>6s}
    {r[4]}")
print()
for k, v in tally.most_common():
    print(f" {k:12s} {v}")
```

```
.venv/bin/python tools/triage.py /path/to/your/documents | tee triage.txt
```

1.2 The five piles, and what each one means

Pile	Means	Go to
ANSWERED, correct	working	add to corpus as a regression guard (§1.3)
ANSWERED, WRONG doctype	urgent	Part 4
ABSTAINED	not sure	Part 3 — or Part 2 if the type isn't in the registry
NEEDS_OCR	never reached the classifier	Part 6
TRUNCATED	classified on part of itself	Part 6

Do the wrong ones first, always. One wrong doctype outweighs ten abstentions.

1.3 A document that classified correctly is not “done”

Add it to the corpus. It becomes the thing that tells you when a later change breaks it — and later changes will try to.

```
cp yourdoc.pdf corpus/us/us_w9__variant_bank_abc.pdf
```

```
{
  "file": "corpus/us/us_w9__variant_bank_abc.pdf",
  "expected_doctype": "us_w9",
  "kind": "specimen",
  "notes": "Bank ABC's rendering; different field ordering to the IRS blank."
}
```

Never commit a real person's data. Issuer specimens, synthetic samples, or a real layout with every value replaced — keeping the *printed wording*, which is what the classifier reads. Three files were deleted from this repo for carrying real identifiers.

If a document is too sensitive to commit, keep it outside the repo and measure locally. A measurement you can run beats a corpus entry you cannot share.

Part 2 — Adding a doctype the registry does not have

2.1 First: is it really new?

```
curl -s localhost:8200/api/v1/doctypes | python3 -c "  
import json,sys  
for d in json.load(sys.stdin)['doctypes']:  
    print(d['doctype_id'], '|', d['label'])" | grep -i "utility\|payslip"
```

There are 129. Check for a generic that already covers it — `xx_utility_bill`, `xx_bank_statement`, `xx_photo_id_generic` exist precisely for “this issuer is not modelled”. Adding `acme_bank_statement_2024` when `xx_bank_statement` would do makes the registry worse.

Add a specific doctype when: the fields you need differ, the issuer’s wording is distinctive enough to anchor on, or downstream treats it differently. Otherwise use the generic.

2.2 The anatomy of a doctype

From the real registry:

```

DocTypeSpec(
  doctype_id="us_passport",          # snake_case, country-prefixed
  label="US Passport (book)",        # what a human sees
  country="US",                      # US | CA | MX | XX
  category=Category.identity,        #
  identity|address_proof|tax|financial|corporate|other
  issuing_authority="U.S. Department of State",
  officially_valid=True,             # counts as KYC proof of
  identity
  applies_to="individual",           # individual | corporate

  anchors=[
    _a("P<USA", decisive=True, controls=Controls.MRZ_PREFIX),
    _a("United States of America", zone=Zone.title),
    _a("PASSPORT", zone=Zone.title),
    _a("Department of State"),
  ],
  id_patterns=[MRZ_TD3_USA],

  confusable_with={
    "us_passport_card": "the card is TD1, MRZ starts I<USA, titled
    PASSPORT CARD",
    "ca_passport": "Canadian MRZ starts P<CAN and the data page is
    bilingual",
  },
  negative_anchors=["PASSPORT CARD", "PASSEPORT", "PASAPORTE"],

  fields=[...],                     # Part 5
)

```

2.3 Choosing anchors — the part that decides everything

Anchors are printed issuer text, not the values on the document. Read your specimen and write down what is printed on *every* instance of that document regardless of who it is about.

NON-DECISIVE ANCHORS — MOST OF THEM

```
_a("Department of the Treasury")
_a("Request for Taxpayer Identification Number", zone=Zone.title)
_a("Recibo de Luz", lang="es")
```

These accumulate evidence. Add 4–10. Be generous; they are cheap and they cannot cause a wrong answer on their own.

DECISIVE ANCHORS — NEAR-PROOF, AND HEAVILY POLICED

A decisive anchor must declare **why** it is decisive, and the loader refuses the pack if it does not:

```
_a("OMB No. 1545-0074", decisive=True, controls=Controls.FORM_NUMBER)
```

The property required is exact:

A decisive anchor must not appear on a document of another type — **including being cited by one.**

Controls	Use for	Strength
FORM_NUMBER	a numbered form from one authority — OMB No. 1545-0074	strong
CONTROL_NUMBER	a printed control/reference number	strong
MRZ_PREFIX	ICAO 9303 prefix — P<USA, I<CAN	strong
STATUTE_TITLE	statutory title of an instrument	medium
ISSUER_NAME	the issuing body as printed	medium
ISSUER_TEMPLATE	verbatim wording from the issuer's own template	medium
CLASS_NAME_UNCONTESTED	known-weak — a class name nothing currently collides with	weak

`CLASS_NAME_UNCONTESTED` is documented in the registry as *the tier of no evidence yet*. The loader forbids any other doctype declaring the same string, so the pack fails to import the moment two collide. **That import failure is the design working — do not work around it.**

Note also: **fuzzy matching is refused for decisive anchors.** A near-proof claim must be seen, not approximated.

ZONES — READ THIS BEFORE USING ONE

```
_a("DRIVER LICENSE", decisive=True,
    controls=Controls.CLASS_NAME_UNCONTESTED, zone=Zone.title)
```

A zone-gated anchor **only fires on a reading that has zones, and only `azure_layout` produces them.** PDF text layers, local OCR and `azure_read` are all zone-free — every block is `Zone.body`.

Measured consequence: `us_drivers_license`, `ca_drivers_license` and `ca_nexus` have *every* decisive anchor title-gated, so on any other path **they cannot be decisively identified at all**. Gating an anchor to a zone is a decision that the doctype requires DI. Make it deliberately.

NEGATIVE ANCHORS AND CONFUSABLES

```
negative_anchors=["PASSPORT CARD", "PASSEPORT"], # these argue AGAINST
this doctype
confusable_with={"us_passport_card": "the card is TD1 and titled PASSPORT
CARD"},
```

`confusable_with` does not affect scoring — it is documentation that appears in the console and tells the next person what separates two lookalikes. Write it as a sentence, not a keyword.

2.4 The checklist

1. Add the `DocTypeSpec` to the right pack.

2. Add a corpus document + manifest line.
3. Add a test in `tests/test_registry_<country>.py`.
4. **Run the whole corpus.** A new doctype changes the IDF profile and can move documents that have nothing to do with it. This is not theoretical — it is why the run is mandatory.

```
.venv/bin/python -m pytest -q  
.venv/bin/python tools/corpus_test.py --ingest --url  
http://localhost:8200 --verbose
```

Part 3 — A document abstained and should not have

Work in this order. Each step rules out a class of cause.

3.1 Are zones the problem?

If `text_source` is `native_text`, `local_ocr` or `azure_read`, no title-gated anchor could fire. **Re-run through `azure_layout` before changing anything.** This is the single most common cause of a photo ID abstaining, and it is a configuration answer, not a registry one.

3.2 Read which gate failed

```
docker logs dce 2>&1 | grep 'classify.abstain'
```

```
classify.abstain declined=us_w9 declined_score=0.61 confidence=0.61
margin=0.02 coverage=0.44 reason=<the gate that failed>
```

Symptom	Cause	Fix
coverage low	the document's vocabulary barely overlaps the doctype's profile	add anchors from the <i>actual</i> printed wording of your specimen
margin small, right doctype leading	two doctypes genuinely confusable	add a discriminating anchor present on one and absent on the other; record it in <code>confusable_with</code>
confidence low, nothing leading	almost no anchors matched	your specimen's wording differs from the registry's assumptions — read it and add what is actually printed

3.3 The fix is almost always an anchor

```
# before
anchors=[_a("Utility Bill"), _a("Account Number")]

# after – words this issuer actually prints
anchors=[
    _a("Utility Bill"),
    _a("Account Number"),
    _a("Comisión Federal de Electricidad", lang="es"),
    _a("Recibo de Luz", lang="es", zone=Zone.title),
    _a("Aviso-Recibo", lang="es"),
    _a("Total a pagar", lang="es"),
]
```

3.4 What not to do

Do not lower a threshold. They are what hold precision at 100%. Three attempts to tune this system have been made, measured, and reverted for costing precision. If you lower one, the corpus run must show precision held — and it usually will not.

Part 4 — A document was classified WRONG

Urgent. Highest severity in the system.

4.1 Find the anchor that did it

The console's evidence panel names every anchor that fired and its weight. Find one belonging to the wrong doctype.

4.2 If a DECISIVE anchor fired wrongly, that anchor is the bug

Decisive means near-proof. If it is not, it must be demoted or narrowed:

```
# was — matched Canadian PR cards too
_a("PERMANENT RESIDENT CARD", decisive=True,
  controls=Controls.CLASS_NAME_UNCONTESTED)

# now — non-decisive; it accumulates evidence but cannot decide alone
_a("PERMANENT RESIDENT CARD")
```

That is a real precedent from this repo.

Options in order of preference:

1. **Demote** to non-decisive.
2. **Narrow** — replace a class name with a form or control number.
3. **Negative anchor** on the winner: `negative_anchors=["PERMIS DE CONDUIRE"]`.
4. **Record it** in `confusable_with` regardless of which you chose.

4.3 Then re-run everything

Anchors interact. A change that fixes one document routinely costs two others, and the only way to know is the corpus run. **Never ship an anchor change without a before/after.**

Part 5 — Extraction

Extraction runs only after classification succeeded, against the fields on that doctype. If classification is wrong, fix that first — extracting against the wrong schema is meaningless.

5.1 The anatomy of a field

```
FieldSpec(
    name="passport_number",          # snake_case
    attribute_key="id.passport_number", # maps into the fleet ontology
    type="id",                       #
    string|date|number|name|address|id|bool
    required=True,                   # missing → needs_review
    pii=True,                        # redacted in the console's raw
    panel
    multi=False,                     # True if several values are
    legitimate
    labels={"en": ["Passport No.", "Passport Number", "Document No."]},
    pattern=r"^[A-Z0-9]{9}$",        # value shape; also REJECTS a
    wrong binding
    validator="mrz_td3",              # named validator, below
    locators=["mrz", "kv", "label"], # tried in this order
    notes="US passport numbers are 9 characters.",
)
```

5.2 The six locators

`FieldSpec.locators` names them in priority order. First one to produce a candidate wins.

Locator	Finds a value by	Best for	Needs
<code>kv</code>	provider-detected key/value pairs	forms	<code>azure_layout</code> — local paths produce no key/values
<code>label</code>	text next to a label from <code>labels</code>	most fields	any path
<code>table</code>	a cell in a detected table	statements, schedules	a provider that detects tables
<code>mark</code>	a checkbox state	tick-boxes	<code>azure_layout</code>
<code>regex</code>	<code>pattern</code> matched anywhere on the document	distinctive shapes (RFC, EIN)	any path
<code>mrz</code>	the machine-readable zone	passports, ID cards	a legible MRZ

Order matters. Put the most precise first. `["kv", "label", "regex"]` is a good default: try the structured answer, then the labelled one, then the shape.

5.3 The fourteen validators

`validator="name"` runs a named check from `dce/extract/validate.py`. A field that fails validation is not bound — which is how a locator that matched the wrong text gets rejected.

```
address  amount  curp   ein   generic_date  iso_date  itin
mrz_td1  mrz_td2  mrz_td3  name  rfc    sin_luhn  ssn
```

Several are **checksum** validators (`sin_luhn`, `curp`, `rfc`, the MRZ family). Those are the strongest thing you can attach to a field: a value that passes a checksum is almost certainly right, and one that fails is almost certainly a mis-binding.

To add a validator:

```
# dce/extract/validate.py
@register("my_scheme")
def my_scheme(value: str, context=None) -> ValidationResult:
    digits = _digits(value)
    if len(digits) != 11:
        return _fail("expected 11 digits")
    return ValidationResult(ok=True, normalized=digits)
```

5.4 Recipes

A FIELD COMES BACK EMPTY

1. Is the label right? Print what the document actually says and add it to `labels`. Labels are per-language: `{"en": [...], "es": [...]}`.
2. Is the locator right? A form field on `azure_layout` should try `kv` first. A value with a distinctive shape should have `regex` with a `pattern`.
3. Is the pattern too tight? A `pattern` rejects a binding that does not match — good for precision, but a real value that fails it disappears silently.
4. Is the value on a page the segmentation put in a different segment? On a bundle, extraction runs per segment against that segment's pages only.

A FIELD BINDS THE WRONG VALUE

Usually the locator matched a caption rather than a value, or a neighbouring field.

1. **Tighten `pattern`**. This is the strongest single fix — it rejects the wrong shape outright.
2. **Add a `validator`** if the value has a checksum. A wrong bind almost never passes one.
3. **Reorder `locators`** to put the precise one first.

Real precedent: a `type="bool"` field returned `"Driver's License – Over 21"` — a title fragment where a yes/no belonged. Fixed with a boolean coercion; the general lesson is that a value which is obviously a *label* means the locator matched the caption.

ADDING A FIELD TO AN EXISTING DOCTYPE

```
fields=[
    ...,
    FieldSpec(
        name="account_number",
        attribute_key="account.number",
        type="id",
        required=False,          # start False; make it required once it
reliably fills
        pii=True,                # any customer identifier
        labels={"en": ["Account Number", "Account No.", "A/C No."]},
        pattern=r"^[0-9]{6,17}$",
        locators=["kv", "label", "regex"],
    ),
]
```

Start `required=False`. A required field that does not fill sends every document to review, which looks like a regression to everyone downstream. Promote it once the fill rate justifies it.

TESTING EXTRACTION

```
curl -s localhost:8200/api/v1/extract \
  -H 'content-type: application/json' \
  -d '{"doctype_id":"us_w9","ingest":
{"filename":"x.pdf"},"content_base64":"'$(base64 -i x.pdf)'"}' \
  | python3 -m json.tool
```

Pinning `doctype_id` skips classification, so you are testing extraction alone. `/extract` never runs a paid tier — it is the surface to call in a loop while tuning locators.

Part 6 — Reading problems

The document came back `needs_ocr`

1. **Is a recogniser configured?** `/readyz` → `ocr.configured_providers`.
2. **Did the request decline it?** The console sends `local_ocr: false` when the channel toggle is on **lexical**, and `?read=lexical` is **sticky in the URL** across reloads and documents.
3. **The prefix mistake:** `AZURE_DI_ENDPOINT` configures the *extraction tiers*; ingest OCR needs `DCE_INGEST_AZURE_DI_ENDPOINT`.

The document came back `truncated`

Check `limits_hit`:

Cap	Means	Setting
<code>unread_pages</code>	pages had no text and nothing could read them	configure a recogniser
<code>max_pages</code>	document longer than the page cap	<code>DCE_INGEST_MAX_PAGES</code> (200)
<code>max_ocr_pages</code>	local engine stopped short	<code>DCE_INGEST_MAX_OCR_PAGES</code> (10)
<code>max_blocks</code> / <code>max_chars</code>	OCR-service response capped	the block/char caps

Policy

`DCE_INGEST_TEXT_LAYER_POLICY=verify`

`# trust | verify | always_ocr`

`always_ocr` if your corpus is overwhelmingly scans and you do not trust text layers. It bills a recognition for every document, including born-digital ones, and refuses to boot without a recogniser.

Part 7 — The measurement loop, which is not optional

```
# 1. BEFORE — note the exact command
.venv/bin/python tools/corpus_test.py --ingest --url
http://localhost:8200 --verbose

# 2. change something

# 3. tests
.venv/bin/python -m pytest -q && .venv/bin/ruff check dce/ tests/

# 4. REBUILD — the container mounts no source
docker compose build dce && docker compose up -d --force-recreate dce

# 5. verify the rebuild took
docker exec dce python3 -c "import dce.ingest.pdf as m;
print(hasattr(m, 'PageVerdict'))"

# 6. AFTER — the SAME command as step 1
```

Read the numbers in this order:

1. **Precision** (`precision_when_answered`) — if it dropped, revert, whatever else improved
2. **Wrong count** — zero is the target and the current state
3. **Accuracy / abstention** — the ambition, not the guarantee

Current baseline: **97 correct / 0 wrong / 20 abstained, 100% precision** on 117 documents.

Two traps that have already cost time: `--ingest` is required or the harness reads PDFs itself and never exercises the code you changed; and the container mounts no source, so a host edit reaches nothing until you rebuild.

Part 8 — Reference

Categories

identity · address_proof · tax · financial · corporate · other

Field types

string · date · number · name · address · id · bool

Zones

title · heading · table · body · furniture Weights: title 3.0× · heading 2.0× ·
table 1.2× · body 1.0× · furniture 0.25× **Only** azure_layout produces anything but
body.

Traps

Trap	Symptom	Fix
<code>?read=lexical</code> sticky	everything needs_ocr	switch to <code>auto</code>
<code>AZURE_DI_ENDPOINT</code> vs <code>DCE_INGEST_AZURE_DI_ENDPOINT</code>	picker greyed, T2/T3 fine	use the prefixed one
Container mounts no source	change does nothing	rebuild
<code>--ingest</code> omitted	before/after identical	always pass it
Quoted env-file values	flag ignored	never quote in an env-file
Zone-gated anchor off DI	photo ID abstains	use <code>azure_layout</code>
Bundle in <code>corpus/<cc>/manifest.jsonl</code>	precision figure drops	use <code>corpus/bundles/</code>
<code>required=True</code> on a new field	everything goes to review	start <code>False</code>
<code>get_texttrace()</code>	interpreter aborts after ~130 pages	never use it

Part 9 — Order of work, for a large pile

1. **Triage everything** (§1.1). Do not start with the interesting document; start with the distribution.
2. **Fix every wrong answer.** However few, they outrank everything.
3. **Add corpus entries for everything that worked.** This is the cheapest work with the highest long-term value, and it is what makes step 4 safe.
4. **Group the abstentions by doctype.** Ten abstentions of one type is one anchor fix, not ten problems. This is where the leverage is.
5. **Add the genuinely new doctypes**, most frequent first.
6. **Only then extraction.** Coverage before depth: a doctype that classifies but extracts nothing is more useful than one that does neither.
7. **Re-measure, and write down what changed.** The number is the deliverable, not the diff.

Two things worth knowing before you start

Ten abstentions of one doctype is usually one missing anchor. Group before you fix.

The corpus is your regression suite and it is currently thin — 84 of 129 doctypes have an example, and it holds no real bundles and no genuinely mixed documents. Every real document you add makes every future change safer. That is the highest-value habit in this whole manual.